

Тестовые случаи и регрессионная проверка

Группа: 307.

Продолжение темы: проверка работоспособности после рефакторинга программного кода.

Цель: научиться выбирать тестовые данные, находить изменения поведения программы, оформлять результаты и проверять исправления.

Необходимые знания: функции, классы, объекты, методы, конструктор `__init__()`, переопределение методов и наследование.

Этап	Время
Проблемный вопрос и обсуждение	5 минут
Лекция	15 минут
Практическая работа	35 минут
Обсуждение результатов	5 минут

Проблемный вопрос

Разработчик сократил функцию. Сохранилось ли её поведение? Какие значения возраста нужно проверить?

До рефакторинга:

```
def can_enter(age):  
    if age >= 18:  
        return "Доступ разрешён"  
    else:  
        return "Доступ запрещён"
```

После рефакторинга:

```
def can_enter(age):  
    if age > 18:  
        return "Доступ разрешён"  
  
    return "Доступ запрещён"
```

Сравните ответы для 17, 18 и 19 лет. Объясните, почему два успешных запуска ещё не доказывают сохранение поведения программы.

1. Что такое тестовый случай

Тестовый случай, или тест-кейс, - описание конкретной проверки: что проверяем, какие данные используем и какой результат должны получить.

Например, в учебной системе колледжа ЕКЕВ зачёт получают студенты, набравшие не менее 50 баллов.

Поле	Содержание
Название	Зачёт при минимальном проходном балле
Входные данные	50 баллов
Действие	Вызвать функцию определения результата
Ожидаемый результат	«Зачёт»

После выполнения проверки добавляют фактический результат и статус: «Пройдена» или «Не пройдена».

Ожидаемый результат определяют до запуска программы на основании требований. Нельзя просто переписать ответ программы в графу «Ожидаемый результат».

2. Откуда брать ожидаемый результат

Главный источник - требования: описание того, как программа должна работать. Например: одна страница стоит 30 тенге; при печати от 10 страниц включительно действует скидка 10% на весь заказ.

- 5 страниц: $5 \times 30 = 150$ тенге.
- 10 страниц: $10 \times 30 \times 0,9 = 270$ тенге.
- 20 страниц: $20 \times 30 \times 0,9 = 540$ тенге.

Если старая и новая версии возвращают одинаковый ответ, это ещё не доказывает правильность: обе версии могут содержать одну и ту же ошибку.

3. Как выбирать данные для проверки

Данные разделяют на группы с похожей обработкой. Такие группы называют **классами эквивалентности**. Для зачёта при допустимых баллах от 0 до 100:

Группа	Ожидаемое поведение	Пример
0-49	Незачёт	30
50-100	Зачёт	75
Меньше 0	Ошибка ввода, если предусмотрена требованиями	-1
Больше 100	Ошибка ввода, если предусмотрена требованиями	101

Представители групп позволяют проверить основную логику. Дополнительно проверяют **граничные значения**: 49, 50, 51. Если функция проверяет допустимый диапазон, добавляют -1, 0, 100, 101.

4. Что проверять, кроме числа на экране

- **Значение:** правильная ли сумма получилась?
- **Тип данных:** вернулось число или строка?
- **Возврат:** функция вернула значение или только вывела его?
- **Изменение состояния:** не изменилась ли посторонняя переменная?
- **Повторные вызовы:** сохраняется ли правильное поведение при повторном использовании?

Функции ниже возвращают разные типы данных:

```
def payment_number():  
    return 5000
```

```
def payment_text():  
    return "5000"
```

```
result = payment_text()  
print(result)  
print(type(result))
```

Вторая функция возвращает строку. Её нельзя напрямую использовать как число для дальнейших вычислений. Команда `type()` позволяет посмотреть тип результата.

5. Как проводить регрессионную проверку

Регрессионная проверка помогает убедиться, что изменения не нарушили ранее работавшие возможности.

1. Подготовить тестовые случаи по требованиям.
2. Выполнить их на исходной версии.
3. Внести изменение в код.
4. Повторить те же проверки.
5. Сравнить фактические результаты с ожидаемыми.
6. Проверить связанные функции.

Повторная проверка найденной ошибки после исправления **подтверждает исправление**. Выполнение остальных важных проверок помогает обнаружить побочные ошибки и относится к регрессионной проверке.

6. Как описывать найденную ошибку

Сообщение «программа работает неправильно» не позволяет воспроизвести проблему. Укажите действие, ожидаемый и фактический результат.

Поле	Пример описания
Ошибка	Скидка не применяется при заказе ровно 10 страниц
Действие	Вызвать <code>print_cost(10)</code>
Ожидаемый результат	270 тенге
Фактический результат	300 тенге

Практическая работа

Ситуация: вы тестируете учебный сервис колледжа ЕКЕВ. Разработчик изменил несколько функций. Проверьте, сохранилась ли их работоспособность.

Для каждого задания: запишите ожидаемые результаты до запуска; выполните проверки; зафиксируйте фактические результаты; объясните и исправьте ошибку; повторите все подготовленные проверки.

Задание 1. Результат зачёта • лёгкое

Требование: функция принимает целое число от 0 до 100. При результате 50 баллов и выше возвращает «Зачёт», иначе - «Незачёт». Проверка некорректного ввода не требуется.

До рефакторинга:

```
def result_before(score):
    if score >= 50:
        answer = "Зачёт"
    else:
        answer = "Незачёт"

    return answer
```

После рефакторинга:

```
def result_after(score):
    if score > 50:
        return "Зачёт"

    return "Незачёт"
```

Выполните: проверьте 0, 49, 50, 51, 100; сравните версии; укажите значение, обнаруживающее ошибку; исправьте новую версию.

```
score = 50
print("До:", result_before(score))
print("После:", result_after(score))
```

№	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____

Вопрос: почему проверки только со значениями 20 и 80 не обнаружат ошибку?

Задание 2. Стоимость печати • среднее

Требования: одна страница стоит 30 тенге. При заказе от 10 страниц включительно действует скидка 10% на весь заказ. При нулевом количестве стоимость равна нулю. Количество страниц - целое неотрицательное число.

До рефакторинга:

```
def print_cost_before(pages):  
    total = pages * 30  
  
    if pages >= 10:  
        total = total * 0.9  
  
    return total
```

После рефакторинга:

```
def print_cost_after(pages):  
    if pages >= 10:  
        return pages * 30 - 0.1  
  
    return pages * 30
```

- Проверьте значения 0, 1, 9, 10, 11.
- Рассчитайте ожидаемые суммы вручную.
- Найдите ошибку в расчёте скидки и исправьте функцию.
- Самостоятельно добавьте ещё одну проверку заказа со скидкой.
- Повторите проверки после исправления.

№	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____
6	_____	_____	_____	_____

Вопрос: чем отличаются «уменьшить сумму на 10%» и «вычесть из суммы 0,1»?

Краткое описание обнаруженной ошибки

Действие / входные данные: _____

Ожидаемый результат: _____

Фактический результат: _____

Задание 3. Число есть, результата нет • лёгкое

Требование: функция должна возвращать стоимость покупки, чтобы программа могла использовать её в дальнейших вычислениях.

До рефакторинга:

```
def purchase_before(price, quantity):  
    return price * quantity
```

После рефакторинга:

```
def purchase_after(price, quantity):  
    print(price * quantity)
```

Проверка:

```
old_result = purchase_before(700, 2)  
new_result = purchase_after(700, 2)  
  
print("Старая версия вернула:", old_result)  
print("Новая версия вернула:", new_result)
```

- До запуска предположите, что появится на экране.
- Запустите программу и объясните результат.
- Исправьте новую функцию.
- После исправления рассчитайте остаток от 5000 тенге.

```
balance = 5000 - purchase_after(700, 2)  
print("Остаток:", balance)
```

№	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	_____	_____	_____	_____
2	_____	_____	_____	_____

Вопрос: почему вывод суммы на экран не заменяет возврат значения?

Ваше объяснение

Задание 4. Регрессионная проверка • среднее

Требование: функция принимает целое количество дней просрочки возврата книги. При отрицательном значении возвращает «Ошибка»; от 0 до 3 дней включительно штраф равен нулю; более 3 дней - по 100 тенге за каждый день сверх первых трёх.

До рефакторинга:

```
def fine_before(days):  
    if days < 0:  
        return "Ошибка"  
    elif days <= 3:  
        return 0  
    else:  
        return (days - 3) * 100
```

После рефакторинга:

```
def fine_after(days):  
    if days <= 3:  
        return 0  
  
    return days * 100
```

- Выберите не менее пяти входных значений.
- Включите отрицательное значение, отсутствие просрочки, границу бесплатного периода и значение сразу после неё.
- Заполните таблицу проверок. Найдите две ошибки.
- Исправьте новую версию и повторите все проверки.
- Оформите краткое описание одного дефекта.

№	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____
6	_____	_____	_____	_____

Описание дефекта

Название: _____

Действие: _____

Ожидаемый результат: _____

Фактический результат: _____

Задание 5. Рефакторинг классов и наследования • повышенное

Требование: базовый класс Student хранит имя и количество баллов. Метод get_result() возвращает «Зачёт» при 50 баллах и выше. Класс GrantStudent наследуется от Student и переопределяет get_result(): для сохранения гранта требуется не менее 70 баллов.

До рефакторинга:

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def get_result(self):
        if self.score >= 50:
            return "Зачёт"
        return "Незачёт"

class GrantStudent(Student):
    def get_result(self):
        if self.score >= 70:
            return "Грант сохранён"
        return "Грант не сохранён"
```

После рефакторинга:

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def get_result(self):
        if self.score > 50:
            return "Зачёт"
        return "Незачёт"

class GrantStudent(Student):
    pass
```

- Создайте объекты Student с баллами 49, 50 и 51.
- Создайте объекты GrantStudent с баллами 69, 70 и 71.
- Сравните поведение до и после рефакторинга.
- Найдите две ошибки, связанные с границей и наследованием.
- Исправьте классы и повторите проверки.
- Объясните, почему pass в дочернем классе изменил поведение программы.

№	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____

№	Входные данные	Ожидаемый результат	Фактический результат	Статус
5	_____	_____	_____	_____
6	_____	_____	_____	_____

Дополнительно: проверьте, что имя и баллы объекта корректно сохраняются конструктором и доступны методам дочернего класса.

Что нужно сдать

- Файл Python с исправленными функциями и классами.
- Таблицы проверок по четырём заданиям.
- Описание одного найденного дефекта.
- Вывод из 2-3 предложений: какие ошибки обнаружены и какими проверками подтверждено исправление.

Для фиксации повторных проверок добавьте в свою таблицу графы «Результат после исправления» и «Повторный статус» либо составьте вторую таблицу.

Критерии оценивания • 10 баллов

Критерий	Баллы
Выбраны обычные и граничные значения	2
Правильно определены ожидаемые результаты	2
Найдены и объяснены ошибки	2
Код исправлен, проверки повторены	2
Оформлены таблицы, дефект и вывод	2

Дополнительное задание

Напишите функцию `check_result(actual, expected)`, которая выводит «Пройдена» или «Не пройдена». Используйте её для проверки исправленных функций.

Вопросы для обсуждения

1. Почему ожидаемый результат нужно определить до запуска программы?
2. Почему при проверке условия $\geq$ особенно важно значение самой границы?
3. Чем подтверждение исправления отличается от регрессионной проверки?
4. Если после исправления одна проверка прошла, почему нужно повторить остальные?

Вывод по работе
